

Task 2 - Kubernetes Cluster Setup

RKE2 Cluster | Worker Node | Rancher | Nginx Deployment | Scaling

Trainee	Atul Kumar
Task Name	Kubernetes Cluster Setup
Technologies	Kubernetes, RKE2, Rancher, Helm, cert-manager, Nginx
Date	May 2026
Master Node	atulrana - 192.168.122.170
Worker Node	postgres-node3 - 192.168.122.193
Kubernetes Version	v1.35.5+rke2r1
Final Status	Completed and verified in terminal + Rancher UI

1. Objective and Requirement Summary

Requirement	Status	Proof
Install RKE2 cluster on master node	Done	RKE2 server installed and started on master node atulrana
Add worker node	Done	postgres-node3 joined successfully and became Ready
Import/manage cluster in Rancher	Done	Rancher installed in the cluster and the local RKE2 cluster is visible/managed
Deploy nginx application	Done	nginx deployment created successfully
Create deployment and service	Done	Deployment + NodePort service created
Scale application pods	Done	Deployment scaled to 3 replicas and verified from terminal and Rancher UI

Simple definition: RKE2 is a secure Kubernetes distribution. Rancher is a web UI used to manage Kubernetes clusters, nodes, workloads, services, and scaling.

Final flow: Two Ubuntu VMs were prepared. The master node atulrana (192.168.122.170) was configured with RKE2 server, the worker node postgres-node3 (192.168.122.193) was configured with RKE2 agent, the worker was joined to the cluster, nginx was deployed and exposed using NodePort, pods were scaled to 3 replicas, then Rancher was installed and used to verify the cluster and workloads.

2. Final Architecture

Browser / Rancher UI

|

v

https://rancher.192.168.122.170.sslip.io

|

v

RKE2 Ingress Nginx Controller

|

v

Rancher Service + Rancher Pod

|

v

Local RKE2 Kubernetes Cluster

|

+-- Master Node: atulrana 192.168.122.170

| Roles: control-plane, etcd

|

+-- Worker Node: postgres-node3 192.168.122.193

Role: worker

Application Flow:

User Browser -> NodePort 31768 -> nginx-app Service -> 3 nginx Pods

3. Environment Details

Node	Hostname	IP Address	OS	Role
Node 1	atulrana	192.168.122.170	Ubuntu 24.04.4 LTS	Master / Control Plane / etcd
Node 2	postgres-node3	192.168.122.193	Ubuntu 24.04.4 LTS	Worker / Agent

4. Pre-Checks and Hostname Resolution

Step 4.1 - Verify hostname/IP details

Command:

```
hostnamectl
hostname -I
```

What this does: Used to confirm each VM hostname and IP address before cluster setup.

Result: Master IP: 192.168.122.170. Worker IP: 192.168.122.193.

Step 4.2 - Update /etc/hosts on both nodes

Command:

```
sudo nano /etc/hosts
```

What this does: Both nodes were mapped to each other by hostname so Kubernetes/RKE2 node communication remains stable.

Result: Entries used: 192.168.122.170 atulrana and 192.168.122.193 postgres-node3.

Step 4.3 - Ping test between nodes

Command:

```
ping atulrana
ping postgres-node3
```

What this does: Verified that master and worker can resolve and reach each other by hostname.

Result: Ping successful after /etc/hosts correction.

5. Install and Configure RKE2 Server on Master Node

Step 5.1 - Install RKE2 server

Command:

```
curl -sL https://get.rke2.io | sudo sh -
```

What this does: Downloads and installs RKE2 server binaries on the master node.

Result: RKE2 v1.35.5+rke2r1 downloaded and unpacked to /usr/local.

```
atul@atulrana:~$ curl -sL https://get.rke2.io | sudo sh -
[INFO] finding release for channel stable
[INFO] using v1.35.5+rke2r1 as release
[INFO] downloading checksums at https://github.com/rancher/rke2/releases/download/v1.35.5%2Brke2r1/sha256sum-amd64.txt
[INFO] downloading tarball at https://github.com/rancher/rke2/releases/download/v1.35.5%2Brke2r1/rke2.linux-amd64.tar.gz
[INFO] verifying tarball
[INFO] unpacking tarball file to /usr/local
atul@atulrana:~$
```

Command output screenshot

Step 5.2 - Enable RKE2 server service

Command:

```
sudo systemctl enable rke2-server.service
```

Task 2 - Kubernetes Cluster Setup | RKE2 + Rancher

What this does: Configures RKE2 server to start automatically on boot.

Result: Symlink created for rke2-server.service.

```
atul@atulrana:~$ sudo systemctl enable rke2-server.service
Created symlink /etc/systemd/system/multi-user.target.wants/rke2-server.service → /usr/local/lib/systemd/system/rke2-server.service.
```

Command output screenshot

Step 5.3 - Start RKE2 server service

Command:

```
sudo systemctl start rke2-server.service
```

What this does: Starts the Kubernetes control plane, etcd, scheduler, controller-manager, kubelet, and RKE2 components.

Result: Command completed successfully.

```
atul@atulrana:~$ sudo systemctl start rke2-server.service

atul@atulrana:~$
```

Command output screenshot

Step 5.4 - Verify RKE2 server status

Command:

```
sudo systemctl status rke2-server --no-pager
```

What this does: Checks that the RKE2 server service is active and running.

Result: Status: active (running).

```
atul@atulrana:~$ sudo systemctl status rke2-server --no-pager
● rke2-server.service - Rancher Kubernetes Engine v2 (Server)
   Loaded: loaded (/usr/local/lib/systemd/system/rke2-server.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-30 08:52:15 UTC; 48s ago
     Docs: https://github.com/rancher/rke2#readme
   Process: 10472 ExecCondition=bin/sh -c if systemctl is-active --quiet rke2-agent.service; then echo "Error: rke2-agent is running!"; exit 1; fi (code=exited, status=0/SUCCESS)
   Process: 10475 ExecStartPre=sbin/modprobe br_netfilter (code=exited, status=0/SUCCESS)
   Process: 10481 ExecStartPre=sbin/modprobe overlay (code=exited, status=0/SUCCESS)
  Main PID: 10483 (rke2)
    Tasks: 119
   Memory: 1.8G (peak: 1.8G)
      CPU: 34.375s
   CGroup: /system.slice/rke2-server.service
           └─10483 /usr/local/bin/rke2 server
               └─10500 containerd -c /var/lib/rancher/rke2/agent/etc/containerd/config.toml
                   └─10521 kubelet --volume-plugin-dir=/var/lib/kubelet/volumeplugins --file-check-frequency=5s --sync-frequency=30s --cloud-provider=external --config-dir=/var/lib/rancher/rke2/agent/etc/kubelet.conf...
                       └─10570 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 13213a64f8369481156fdb50a16e0a521bfe5ba859757dd45368fef4bea5061 -address /run/k3s/co...
                           └─10580 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 0d27e7b92188f469b2a06fc9d486b90863863b1d34feca672d3a98f17ef3429 -address /run/k3s/co...
                               └─10732 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 30b26e6358190b3fbaed3e600640e05a0d9e1ff90e55b956c129407bdcde -address /run/k3s/co...
                                   └─10813 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id f31310428419ef7a966bf4dba4e9c7a9d5c73c8887ad708c40fcf1868fcc25b6 -address /run/k3s/co...
                                       └─10841 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id ab0468a250f8e2e484ca3b7452552af420813da950e73ce46523a8b3298bdd05 -address /run/k3s/co...
                                           └─10970 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 6a66a25db8c2268f8f39b1a34c4b4658391769a259e3c33445f6d3d095e38 -address /run/k3s/co...
                                               └─11331 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 8ab6cdc5babb24447e0a5a1dc84f9eb2a5c2c54ad9018c35a125212b3f36d0df -address /run/k3s/co...
                                                   └─11350 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-ad9090cdd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 657d5be9852a668cd14a15f363ccb91914a3050edeaa0fe432daaf9fc025cc1 -address /run/k3s/co...

May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Applied manifest at \"/var/lib/ranc...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Applying manifest at \"/var/lib/ranc...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=helm.cattle.io/v1 fieldPath= kind=HelmChart logger=rke2 message="Applying HelmChart from inline spec.c...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Applied manifest at \"/var/lib/ranc...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Applied manifest at \"/var/lib/ranc...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=helm.cattle.io/v1 fieldPath= kind=HelmChart logger=rke2 message="Applying HelmChart from inline spec.c...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Applying manifest at \"/var/lib/ranc...
May 30 08:52:19 atulrana rke2[10483]: times="2026-05-30T08:52:19Z" level=info msg="Event occurred" apiVersion=k3s.cattle.io/v1 fieldPath= kind=Addon logger=rke2/deploy message="Deleting manifest ..ifest type=Normal"
May 30 08:52:28 atulrana rke2[10483]: times="2026-05-30T08:52:28Z" level=info msg="Adding node atulrana-S29Id209 etcd status condition"
Hint: Some lines were ellipsized, use -l to show in full.
atul@atulrana:~$
```

Command output screenshot

Step 5.5 - Configure kubectl on master

Command:

```
mkdir -p ~/.kube
```

```
sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
```

```
sudo chown $USER:$USER ~/.kube/config
```

```
export PATH=$PATH:/var/lib/rancher/rke2/bin
kubectl get nodes
```

What this does: Copies kubeconfig so kubectl can communicate with the RKE2 API server.

Result: Master node atulrana became Ready with roles control-plane,etcd.

```
atul@atulrana:~$ mkdir -p ~/.kube
atul@atulrana:~$ sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
atul@atulrana:~$ sudo chown $USER:$USER ~/.kube/config
atul@atulrana:~$ kubectl get nodes
Command 'kubectl' not found, but can be installed with:
sudo snap install kubectl
atul@atulrana:~$ export PATH=$PATH:/var/lib/rancher/rke2/bin
atul@atulrana:~$ export PATH=$PATH:/var/lib/rancher/rke2/bin
atul@atulrana:~$ kubectl get nodes
NAME        STATUS    ROLES                  AGE     VERSION
atulrana    Ready     control-plane,etcd     3m3s    v1.35.5+rke2r1
atul@atulrana:~$
```

Command output screenshot

Step 5.6 - Get node token for worker join

Command:

```
sudo cat /var/lib/rancher/rke2/server/node-token
```

What this does: RKE2 agents use this token to securely join the server.

Result: Token was generated. Token is masked in this document for security.

```
atul@atulrana:~$ sudo cat /var/lib/rancher/rke2/server/node-token
*** token hidden for security ***
```

Command output screenshot

6. Install RKE2 Agent and Join Worker Node

Step 6.1 - Install RKE2 agent on worker

Command:

```
curl -sfL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sudo sh -
```

What this does: Installs RKE2 agent binaries on worker node postgres-node3.

Result: RKE2 agent downloaded and unpacked successfully.

```
manavi@postgres-node3:~$ curl -sfL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sudo sh -
[INFO] finding release for channel stable
[INFO] using v1.35.5+rke2r1 as release
[INFO] downloading checksums at https://github.com/rancher/rke2/releases/download/v1.35.5%2Brke2r1/sha256sum-amd64.txt
[INFO] downloading tarball at https://github.com/rancher/rke2/releases/download/v1.35.5%2Brke2r1/rke2.linux-amd64.tar.gz
[INFO] verifying tarball
[INFO] unpacking tarball file to /usr/local
```

Command output screenshot

Step 6.2 - Create RKE2 config on worker

Command:

```
sudo mkdir -p /etc/rancher/rke2
sudo nano /etc/rancher/rke2/config.yaml
```

What this does: Creates agent configuration with master URL and cluster token.

Result: server: https://192.168.122.170:9345. Token masked for security.

```
manavi@postgres-node3:~$ sudo mkdir -p /etc/rancher/rke2
manavi@postgres-node3:~$ sudo nano /etc/rancher/rke2/config.yaml
manavi@postgres-node3:~$ cat /etc/rancher/rke2/config.yaml
server: https://192.168.122.170:9345
*** token hidden for security ***

manavi@postgres-node3:~$
```

Command output screenshot

Step 6.3 - Enable/start RKE2 agent

Command:

```
sudo systemctl enable rke2-agent.service
sudo systemctl start rke2-agent.service
sudo systemctl status rke2-agent --no-pager
```

What this does: Enables and starts worker agent service to join the RKE2 cluster.

Result: Status: active (running).

```
manavi@postgres-node3:~$ sudo systemctl enable rke2-agent.service
Created symlink /etc/systemd/system/multi-user.target.wants/rke2-agent.service → /usr/local/lib/systemd/system/rke2-agent.service.
manavi@postgres-node3:~$ sudo systemctl start rke2-agent.service
manavi@postgres-node3:~$ sudo systemctl status rke2-agent --no-pager
● rke2-agent.service - Rancher Kubernetes Engine v2 (agent)
   Loaded: loaded (/usr/local/lib/systemd/system/rke2-agent.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-30 09:01:31 UTC; 13s ago
     Docs: https://github.com/rancher/rke2#readme
   Process: 19485 ExecCondition=bin/sh -c if systemctl is-active --quiet rke2-server.service; then echo "Error: rke2-server is running!"; exit 1; fi (code=exited, status=0/SUCCESS)
   Process: 19487 ExecStartPre=sbin/modprobe br_netfilter (code=exited, status=0/SUCCESS)
   Process: 19489 ExecStartPre=sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Main PID: 19491 (rke2)
    Tasks: 56
   Memory: 1.0G (peak: 1.0G)
     CPU: 18.639s
   CGroup: /system.slice/rke2-agent.service
           └─19491 /usr/local/bin/rke2 agent
               └─19509 containerd -c /var/lib/rancher/rke2/agent/etc/containerd/config.toml
                   └─19551 kubelet --volume-plugin-dir=/var/lib/kubelet/volumeplugins --file-check-frequency=5s --sync-frequency=30s --cloud-provider=external --config-dir=/var/lib/rancher/rke2/agent/etc/kubelet.conf...
                       └─19632 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-8dd90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 604aa3083c6aedaf7d09e1ae848f406fd558536340911fee41420fa2eabf8d -address /run/k3s/co...
                           └─19641 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-8dd90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 96981cf8cbff726306e45c3eaa5955aeaa92123b306d020275be95690f951b -address /run/k3s/co...
                               └─19912 /usr/bin/unpigrz -d -c

May 30 09:01:07 postgres-node3 rke2[19491]: time="2026-05-30T09:01:07Z" level=info msg="Pulling image index.docker.io/rancher/hardened-kubernetes:v1.35.5-rke2r1-build20260512"
May 30 09:01:13 postgres-node3 rke2[19491]: time="2026-05-30T09:01:13Z" level=info msg="Server 192.168.122.170:6443@PREFERRED*->ACTIVE from successful dial"
May 30 09:01:13 postgres-node3 rke2[19491]: time="2026-05-30T09:01:13Z" level=info msg="Event occurred" apiVersion= fieldPath= kind=Node logger=rke2 message="Node and Certificate Authority cert_LionOK type=Normal"
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=error msg="Failed to process image event: failed to import /var/lib/rancher/rke2/agent/images/kube-proxy-image.txt: failed to pull tm...
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=info msg="Pulling images from /var/lib/rancher/rke2/agent/images/runtime-image.txt"
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=info msg="Pulling image index.docker.io/rancher/rke2-runtime:v1.35.5-rke2r1"
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=info msg="Running kubelet --alsologtostderr=false --cloud-provider=external --config-dir=/var/lib/rancher/rke2/agent/etc/kubelet.conf...
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=info msg="Annotations and labels have been set successfully on node: postgres-node3"
May 30 09:01:31 postgres-node3 rke2[19491]: time="2026-05-30T09:01:31Z" level=info msg="rke2 agent is up and running"
May 30 09:01:31 postgres-node3 systemd[1]: Started rke2-agent.service - Rancher Kubernetes Engine v2 (agent).
Hint: Some lines were ellipsized, use -l to show in full.
manavi@postgres-node3:~$
```

Command output screenshot

Step 6.4 - Verify node registration

Command:

```
kubectl get nodes -o wide
```

What this does: Checks whether the worker joined the master cluster.

Result: Worker registered, but initially showed NotReady due to storage issue. This was resolved in the next section.

```
atul@atulrana:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES          AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
atulrana      Ready     control-plane,etcd  10m   v1.35.5+rke2r1  192.168.122.179  <none>         Ubuntu 24.04.4 LTS   6.8.0-124-generic  containerd://2.2.3-k3s1
postgres-node3 NotReady   <none>          55s   v1.35.5+rke2r1  192.168.122.193  <none>         Ubuntu 24.04.4 LTS   6.8.0-117-generic  containerd://2.2.3-k3s1
```

Command output screenshot

7. Troubleshooting: Worker NotReady Due to Disk Full

Issue: The worker node joined the cluster but stayed NotReady. RKE2 agent logs showed image import/unpack failure with no space left on device. This was a real setup issue, so it is documented here with the clean commands used to resolve it.

Step 7.1 - Check RKE2 agent logs

Command:

```
sudo journalctl -u rke2-agent -n 30 --no-pager
```

What this does: Reviewed worker agent logs to find the root cause of NotReady state.

Result: Error found: no space left on device while importing RKE2 runtime image.

```
manav1@postgres-node3:~$ sudo systemctl restart rke2-agent
manav1@postgres-node3:~$ sudo journalctl -u rke2-agent -n 30 --no-pager
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=warning msg="Failed to load kernel module nft-expr-counter with modprobe"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Removed kube-proxy static pod manifest"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Runtime image index.docker.io/rancher/rke2-runtime:v1.35.5-rke2r1 bin and charts directories already exist; skipping extract"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Waiting for cri connection: rpc error: code = Unavailable desc = connection error: desc = \"transport: Error while dialing: dial unix /run/k3s/containerd/containerd.sock: connect: connection refused\""
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Logging containerd to /var/lib/rancher/rke2/agent/containerd/containerd.log"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Running containerd -c /var/lib/rancher/rke2/agent/etc/containerd/config.toml"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Getting list of apiserver endpoints from server"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Creating rke2-cert-monitor event broadcaster"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Got apiserver addresses from supervisor: [192.168.122.170:6443]"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Connecting to proxy" url="wss://192.168.122.170:9345/v1-rke2/connect"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Connected to proxy" url="wss://192.168.122.170:9345/v1-rke2/connect"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Remotedialer connected to proxy" url="wss://192.168.122.170:9345/v1-rke2/connect"
May 30 09:06:20 postgres-node3 rke2[20544]: time="2026-05-30T09:06:20Z" level=info msg="Running kube-proxy --cluster-cidr=10.42.0.0/16 --conntrack-max-per-core=0 --conntrack-tcp-timeout-close-wait=0s --conntrack-tcp-timeout-established=0s --healthz-bind-address=127.0.0.1 --hostname-override=postgres-node3 --kubeconfig=/var/lib/rancher/rke2/agent/kubeproxy.kubeconfig --proxy-mode=iptables"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Server 192.168.122.170:9345@RECOVERING*->ACTIVE from successful health check"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="containerd is now running"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Pulling images from /var/lib/rancher/rke2/agent/images/kube-proxy-image.txt"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Image index.docker.io/rancher/hardened-kubernetes:v1.35.5-rke2r1-build20260512 has already been pulled"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Imported docker.io/rancher/hardened-kubernetes:v1.35.5-rke2r1-build20260512"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Tagged docker.io/rancher/hardened-kubernetes@sha256:1dca4101b9cf6e629169b28cddd46a40e4e9281073375faad52798afa8224ba"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Server 192.168.122.170:6443@RECOVERING*->ACTIVE from successful health check"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Imported images from /var/lib/rancher/rke2/agent/images/kube-proxy-image.txt in 63.371545ms"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Pulling images from /var/lib/rancher/rke2/agent/images/runtime-image.txt"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Pulling image index.docker.io/rancher/rke2-runtime:v1.35.5-rke2r1"
May 30 09:06:21 postgres-node3 rke2[20544]: time="2026-05-30T09:06:21Z" level=info msg="Running kubelet --also-logtostderr=false --cloud-provider=external --config-dir=/var/lib/rancher/rke2/agent/etc/kubelet.conf.d --containerd=/run/k3s/containerd/containerd.sock --hostname-override=postgres-node3 --kubeconfig=/var/lib/rancher/rke2/agent/kubelet.kubeconfig --log-file=/var/lib/rancher/rke2/agent/logs/kubelet.log --log-file-max-size=50 --logtostderr=false --node-ip=192.168.122.193 --node-labels= --read-only-port=0 --stderrthreshold=FATAL"
May 30 09:06:33 postgres-node3 rke2[20544]: time="2026-05-30T09:06:33Z" level=error msg="Failed to process image event: failed to import /var/lib/rancher/rke2/agent/images/runtime-image.txt: failed to pull images from /var/lib/rancher/rke2/agent/images/runtime-image.txt: rpc error: code = Unknown desc = failed to pull and unpack image \"docker.io/rancher/rke2-runtime:v1.35.5-rke2r1\": failed to copy: write /var/lib/rancher/rke2/agent/containerd/io.containerd.content.v1.content/ingest/9e58aec9cdc04f1f2eca9f9dcfe2bf5766246ff8653b63a31a9128b3c6cf809/data: no space left on device"
May 30 09:06:40 postgres-node3 rke2[20544]: time="2026-05-30T09:06:40Z" level=info msg="Event occurred" apiVersion= kind=Node logger=rke2 message="Node and Certificate Authority certificates managed by rke2 are OK" object=postgres-node3 reason=CertificateExpirationOK type=Normal
May 30 09:06:40 postgres-node3 rke2[20544]: time="2026-05-30T09:06:40Z" level=info msg="Tunnel authorizer set Kubelet Port 0.0.0.0:10250"
May 30 09:06:40 postgres-node3 systemd[1]: Started rke2-agent.service - Rancher Kubernetes Engine v2 (agent).
May 30 09:06:40 postgres-node3 rke2[20544]: time="2026-05-30T09:06:40Z" level=info msg="Annotations and labels have been set successfully on node: postgres-node3"
May 30 09:06:40 postgres-node3 rke2[20544]: time="2026-05-30T09:06:40Z" level=info msg="rke2 agent is up and running"
manav1@postgres-node3:~$
```

Command output screenshot

Step 7.2 - Check filesystem usage

Command:

```
df -h
```

What this does: Checked root filesystem size and free space.

Result: Root filesystem was 100% full: 9.8G used, 0 available.

```
manav1@postgres-node3:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           291M  1.6M  290M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 9.8G  9.8G   0 100% /
tmpfs           1.5G   0  1.5G   0% /dev/shm
tmpfs           5.0M   0  5.0M   0% /run/lock
/dev/vda2       1.8G  296M  1.4G  19% /boot
tmpfs           291M  12K  291M   1% /run/user/1000
manav1@postgres-node3:~$
```

Command output screenshot

Step 7.3 - Find large directories

Command:


```
sudo du -sh /* 2>/dev/null | sort -hr
```

```
sudo du -sh /var/* 2>/dev/null | sort -hr
```

What this does: Identified directories consuming disk space.

Result: Major usage was /usr and /var/lib.

```
manav1@postgres-node3:~$ sudo du -sh /* 2>/dev/null | sort -hr
4.1G    /usr
3.7G    /var
1.9G    /swap.img
1.3G    /run
296M    /boot
229M    /opt
6.8M    /etc
6.3M    /root
104K    /home
72K     /tmp
16K     /lost+found
8.0K    /snap
4.0K    /srv
4.0K    /sbin.usr-is-merged
4.0K    /mnt
4.0K    /media
4.0K    /lib.usr-is-merged
4.0K    /cdrom
4.0K    /bin.usr-is-merged
0       /sys
0       /sbin
0       /proc
0       /lib64
0       /lib
0       /dev
0       /bin

manav1@postgres-node3:~$ sudo du -sh /var/* 2>/dev/null | sort -hr
3.4G    /var/lib
160M    /var/cache
75M     /var/log
1.2M    /var/backups
52K     /var/tmp
16K     /var/spool
4.0K    /var/snap
4.0K    /var/opt
4.0K    /var/mail
4.0K    /var/local
4.0K    /var/crash
0       /var/run
0       /var/lock

manav1@postgres-node3:~$
```

Command output screenshot

Step 7.4 - Check LVM layout

Command:

```
lsblk
```

What this does: Verified disk, partition, and logical volume sizes.

Result: Disk was 20G and partition was 18.2G, but logical volume was only 10G.


```
manav1@postgres-node3:~$ lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0                                11:0    1 1024M  0 rom
vda                                253:0    0   20G  0 disk
├─vda1                             253:1    0    1M  0 part
├─vda2                             253:2    0   1.8G  0 part /boot
├─vda3                             253:3    0  18.2G  0 part
└─ubuntu--vg-ubuntu--lv           252:0    0   10G  0 lvm /
```

Command output screenshot

Step 7.5 - Resize physical volume

Command:

```
sudo pvresize /dev/vda3
```

What this does: Resized the LVM physical volume to use the full partition.

Result: Physical volume /dev/vda3 changed successfully.

```
manav1@postgres-node3:~$ sudo pvresize /dev/vda3
[sudo] password for manav1:
Physical volume "/dev/vda3" changed
1 physical volume(s) resized or updated / 0 physical volume(s) not resized
manav1@postgres-node3:~$
```

Command output screenshot

Step 7.6 - Check volume group free space

Command:

```
sudo vgdisplay
```

What this does: Displayed volume group size and free space.

Result: Free space available: 8.22 GiB.

[Missing screenshot: /mnt/data/5c3a14b1-c5e2-46b0-874d-6b4f867a07e7.png]

Step 7.7 - Extend logical volume

Command:

```
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
```

What this does: Extended the logical volume using all free space in the volume group.

Result: Logical volume increased from 10.00 GiB to 18.22 GiB.

```
manav1@postgres-node3:~$ sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
Size of logical volume ubuntu-vg/ubuntu-lv changed from 10.00 GiB (2560 extents) to 18.22 GiB (4665 extents).
Logical volume ubuntu-vg/ubuntu-lv successfully resized.
manav1@postgres-node3:~$
```

Command output screenshot

Step 7.8 - Resize filesystem and verify free space

Command:

```
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
df -h
```

What this does: Expanded the ext4 filesystem online and verified new root size.

Result: Root filesystem became 18G with 7.4G free.

```
manav1@postgres-node3:~$ sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/ubuntu-vg/ubuntu-lv is mounted on /; on-line resizing required
old_desc_blocks = 2, new_desc_blocks = 3
The filesystem on /dev/ubuntu-vg/ubuntu-lv is now 4776960 (4k) blocks long.

manav1@postgres-node3:~$ df -h
Filesystem                Size      Used Avail Use% Mounted on
tmpfs                     291M        1.7M  290M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv  18G       9.7G   7.4G  57% /
tmpfs                     1.5G         0   1.5G   0% /dev/shm
tmpfs                     5.0M         0   5.0M   0% /run/lock
/dev/vda2                 1.8G      296M   1.4G  19% /boot
tmpfs                     291M        12K  291M   1% /run/user/1000

manav1@postgres-node3:~$ sudo systemctl restart rke2-agent
manav1@postgres-node3:~$ kubectl get nodes
Command 'kubectl' not found, but can be installed with:
sudo snap install kubectl
manav1@postgres-node3:~$ sudo snap install kubectl
error: This revision of snap "kubectl" was published using classic confinement and thus may perform
arbitrary system changes outside of the security sandbox that snaps are usually confined to.
```

Command output screenshot

Step 7.9 - Restart RKE2 agent and confirm service

Command:

```
sudo systemctl restart rke2-agent
sudo systemctl status rke2-agent --no-pager
```

What this does: Restarted RKE2 agent after disk expansion so image import and kubelet could recover.

Result: RKE2 agent active and running.

Task 2 - Kubernetes Cluster Setup | RKE2 + Rancher

```
manav1@postgres-node3:~$ sudo systemctl status rke2-agent --no-pager
● rke2-agent.service - Rancher Kubernetes Engine v2 (agent)
   Loaded: loaded (/usr/local/lib/systemd/system/rke2-agent.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-05-30 09:25:44 UTC; 1min 31s ago
     Docs: https://github.com/rancher/rke2#readme
   Process: 5335 ExecCondition=/bin/sh -c if systemctl is-active --quiet rke2-server.service; then echo "Error: rke2-server is running!"
   Process: 5338 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, status=0/SUCCESS)
   Process: 5340 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Main PID: 5343 (rke2)
      Tasks: 87
     Memory: 1.7G (peak: 1.7G)
        CPU: 21.019s
     CGroup: /system.slice/rke2-agent.service
            └─1412 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-a0d90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id d1d4206db26a
            └─1414 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-a0d90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 3edbd70769c4
            └─5343 "/usr/local/bin/rke2 agent"
            └─5355 containerd -c /var/lib/rancher/rke2/agent/etc/containerd/config.toml
            └─5423 kubelet --volume-plugin-dir=/var/lib/kubelet/volumeplugins --file-check-frequency=5s --sync-frequency=30s --cloud-pro
            └─5712 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-a0d90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id cb69213f9a8b
            └─5726 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-a0d90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 5e225be4f4f5
            └─5751 /var/lib/rancher/rke2/data/v1.35.5-rke2r1-a0d90cdcd0dc/bin/containerd-shim-runc-v2 -namespace k8s.io -id 7c300e676499

May 30 09:25:25 postgres-node3 rke2[5343]: time="2026-05-30T09:25:25Z" level=info msg="Pulling images from /var/lib/rancher/rke2/agent/im
May 30 09:25:25 postgres-node3 rke2[5343]: time="2026-05-30T09:25:25Z" level=info msg="Pulling image index.docker.io/rancher/rke2-runtime
May 30 09:25:25 postgres-node3 rke2[5343]: time="2026-05-30T09:25:25Z" level=info msg="Server 192.168.122.170:6443@RECOVERING*->ACTIVE fr
May 30 09:25:25 postgres-node3 rke2[5343]: time="2026-05-30T09:25:25Z" level=info msg="Running kubelet --alsologtostderr=false --cloud-pr
May 30 09:25:43 postgres-node3 rke2[5343]: time="2026-05-30T09:25:43Z" level=error msg="Failed to process image event: failed to import /
May 30 09:25:44 postgres-node3 rke2[5343]: time="2026-05-30T09:25:44Z" level=info msg="Event occurred" apiVersion= fieldPath= kind=Node l
May 30 09:25:44 postgres-node3 rke2[5343]: time="2026-05-30T09:25:44Z" level=info msg="Annotations and labels have been set successfully
May 30 09:25:44 postgres-node3 rke2[5343]: time="2026-05-30T09:25:44Z" level=info msg="rke2 agent is up and running"
May 30 09:25:44 postgres-node3 rke2[5343]: time="2026-05-30T09:25:44Z" level=info msg="Tunnel authorizer set Kubelet Port 0.0.0.0:10250"
May 30 09:25:44 postgres-node3 systemd[1]: Started rke2-agent.service - Rancher Kubernetes Engine v2 (agent).
Hint: Some lines were ellipsized, use -l to show in full.
manav1@postgres-node3:~$
```

Command output screenshot

Step 7.10 - Final node readiness check

Command:

```
kubectl get nodes -o wide
```

What this does: Verified both master and worker after fixing storage.

Result: Both nodes became Ready.

```
atul@atulrana:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION        CONTAINER-RUNTIME
atulrana      Ready     control-plane,etcd   35m   v1.35.5+rke2r1   192.168.122.170   <none>         Ubuntu 24.04.4 LTS   6.8.0-124-generic     containerd://2.2.3-k3s1
postgres-node3 Ready     <none>    26m   v1.35.5+rke2r1   192.168.122.193   <none>         Ubuntu 24.04.4 LTS   6.8.0-124-generic     containerd://2.2.3-k3s1

atul@atulrana:~$ export PATH=/var/lib/rancher/rke2/bin
atul@atulrana:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION        CONTAINER-RUNTIME
atulrana      Ready     control-plane,etcd   35m   v1.35.5+rke2r1   192.168.122.170   <none>         Ubuntu 24.04.4 LTS   6.8.0-124-generic     containerd://2.2.3-k3s1
postgres-node3 Ready     <none>    26m   v1.35.5+rke2r1   192.168.122.193   <none>         Ubuntu 24.04.4 LTS   6.8.0-124-generic     containerd://2.2.3-k3s1
atul@atulrana:~$
```

Command output screenshot

8. Deploy Nginx Application

Step 8.1 - Create nginx deployment

Command:

```
kubectl create deployment nginx-app --image=nginx
```

```
kubectl get deployments
```

```
kubectl get pods -o wide
```

What this does: Creates a Kubernetes Deployment. Deployment manages pods and recreates them if they fail.

Result: nginx-app deployment created. Pod scheduled and Running.

```
atul@atulrana:~$ kubectl create deployment nginx-app --image=nginx
deployment.apps/nginx-app created
atul@atulrana:~$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
nginx-app     0/1     1             0           8s
atul@atulrana:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
nginx-app-574b8c4d9c-hz88p         1/1     Running   0           13s   10.42.1.4     postgres-node3   <none>           <none>
```

Command output screenshot

Step 8.2 - Expose nginx deployment using NodePort service

Command:

```
kubectl expose deployment nginx-app --port=80 --target-port=80 --type=NodePort
kubectl get svc
```

What this does: Creates a stable Service in front of the nginx pods. NodePort exposes the app through a port on the cluster nodes.

Result: NodePort assigned: 31768/TCP.

```
atul@atulrana:~$ kubectl expose deployment nginx-app --port=80 --target-port=80 --type=NodePort
service/nginx-app exposed
atul@atulrana:~$ kubectl get svc
NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes    ClusterIP     10.43.0.1     <none>         443/TCP          37m
nginx-app     NodePort      10.43.204.12  <none>         80:31768/TCP     5s
```

Command output screenshot

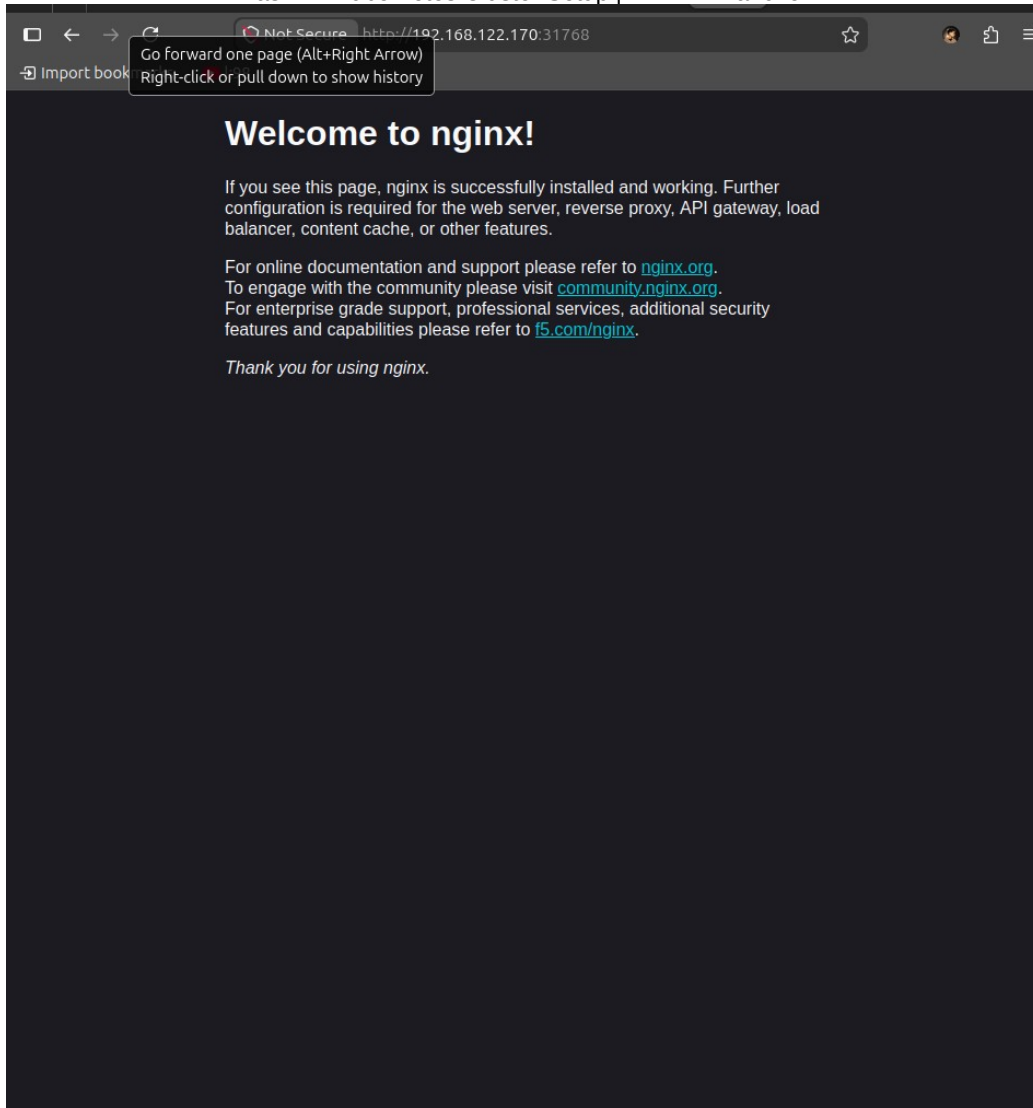
Step 8.3 - Browser access test

Command:

```
http://192.168.122.170:31768
```

What this does: Verified nginx application from browser using NodePort.

Result: Welcome to nginx page loaded successfully.

*Command output screenshot*

Step 8.4 - Scale nginx deployment to 3 replicas

Command:

```
kubectl scale deployment nginx-app --replicas=3
```

What this does: Scaling creates multiple copies of the application pods for availability and load distribution.

Result: Deployment scaled successfully.

```
atul@atulrana:~$ kubectl scale deployment nginx-app --replicas=3
deployment.apps/nginx-app scaled
atul@atulrana:~$
```

Command output screenshot

Step 8.5 - Verify scaled pods

Command:

```
kubectl get pods -o wide
```

What this does: Checks all nginx pods and their node placement.

Result: Three nginx pods running across master/worker nodes.

```
atul@atulrana:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
nginx-app-574b8c4d9c-b6hx5          1/1    Running   0           46s   10.42.1.5       postgres-node3   <none>           <none>
nginx-app-574b8c4d9c-hz88p          1/1    Running   0           4m36s  10.42.1.4       postgres-node3   <none>           <none>
nginx-app-574b8c4d9c-zbq6j          1/1    Running   0           46s   10.42.0.14      atulrana         <none>           <none>
atul@atulrana:~$
```

Command output screenshot

Step 8.6 - Verify all cluster pods

Command:

```
kubectl get pods -A
```

What this does: Shows Kubernetes system pods plus nginx application pods.

Result: Control plane, DNS, metrics, ingress and nginx pods running.

```
atul@atulrana:~$ kubectl get pods -A
NAMESPACE   NAME                                                    READY   STATUS    RESTARTS   AGE
default     nginx-app-574b8c4d9c-b6hx5                            1/1    Running   0           3m9s
default     nginx-app-574b8c4d9c-hz88p                            1/1    Running   0           6m59s
default     nginx-app-574b8c4d9c-zbq6j                            1/1    Running   0           3m9s
kube-system cloud-controller-manager-atulrana                   1/1    Running   0           43m
kube-system etcd-atulrana                               1/1    Running   0           43m
kube-system helm-install-rke2-canal-h7qsv               0/1    Completed 0           43m
kube-system helm-install-rke2-coredns-cqbhb            0/1    Completed 0           43m
kube-system helm-install-rke2-ingress-nginx-s6s9l       0/1    Completed 0           43m
kube-system helm-install-rke2-metrics-server-pn89d      0/1    Completed 0           43m
kube-system helm-install-rke2-runtimeclasses-9rqlz      0/1    Completed 0           43m
kube-system helm-install-rke2-snapshot-controller-crd-rrgmt 0/1    Completed 0           43m
kube-system helm-install-rke2-snapshot-controller-z65fj 0/1    Completed 1           43m
kube-system kube-apiserver-atulrana                     1/1    Running   0           43m
kube-system kube-controller-manager-atulrana             1/1    Running   0           43m
kube-system kube-proxy-atulrana                         1/1    Running   0           43m
kube-system kube-proxy-postgres-node3                  1/1    Running   0           10m
kube-system kube-scheduler-atulrana                     1/1    Running   0           43m
kube-system rke2-canal-5h7cm                            2/2    Running   1 (19m ago)  34m
kube-system rke2-canal-95fvc                            2/2    Running   0           43m
kube-system rke2-coredns-rke2-coredns-69c9c9877c-nrnsc 1/1    Running   0           43m
kube-system rke2-coredns-rke2-coredns-69c9c9877c-qb8lp 1/1    Running   0           18m
kube-system rke2-coredns-rke2-coredns-autoscaler-645c95cdd7-lcd9d 1/1    Running   0           43m
kube-system rke2-ingress-nginx-controller-7ck56        1/1    Running   0           41m
kube-system rke2-ingress-nginx-controller-w9m2h        1/1    Running   0           10m
kube-system rke2-metrics-server-bdc688fb7-wh44x        1/1    Running   0           41m
kube-system rke2-snapshot-controller-6d8cd4bbcc-m7hbt   1/1    Running   0           41m
atul@atulrana:~$
```

Command output screenshot

9. Install Rancher on the RKE2 Cluster

Note: Rancher was installed on the same RKE2 cluster. In Rancher UI this cluster appears automatically as local. This satisfies management visibility in Rancher for the built cluster. If an external Rancher server is required, the same cluster can also be imported using Rancher Import Existing workflow.

Step 9.1 - Install Helm

Command:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

What this does: Helm is a Kubernetes package manager. It is used to install cert-manager and Rancher charts.

Result: Helm v3.21.0 installed successfully.

```
atul@atulrana:~$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 11929 100 11929 0 0 31066 0 --:--:-- --:--:-- --:--:-- 31065
Downloading https://get.helm.sh/helm-v3.21.0-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
atul@atulrana:~$ helm version
version.BuildInfo{Version:"v3.21.0", GitCommit:"e8878d41b711792be60777fd65ad23a101e6b85f", GitTreeState:"clean", GoVersion:"go1.25.10"}
atul@atulrana:~$
```

Command output screenshot

Step 9.2 - Create cert-manager namespace

Command:

```
kubectl create namespace cert-manager
```

What this does: Creates a dedicated namespace for cert-manager components.

Result: Namespace cert-manager created.

```
atul@atulrana:~$ kubectl create namespace cert-manager
namespace/cert-manager created
atul@atulrana:~$
```

Command output screenshot

Step 9.3 - Add Jetstack repo and update

Command:

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
```

What this does: Adds the official cert-manager Helm chart repository and refreshes chart index.

Result: Jetstack repository added and updated.

```
atul@atulrana:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
..Successfully got an update from the "jetstack" chart repository
Update Complete. *Happy Helming!*
atul@atulrana:~$
```

Command output screenshot

Step 9.4 - Install cert-manager

Command:

```
helm install cert-manager jetstack/cert-manager \
--namespace cert-manager \
--set crds.enabled=true
```

What this does: Installs cert-manager and its CRDs. Rancher needs certificates for HTTPS.

Result: cert-manager v1.20.2 deployed successfully.


```

atul@atulrana:~$ helm install cert-manager jetstack/cert-manager \
--namespace cert-manager \
--set crds.enabled=true
NAME: cert-manager
LAST DEPLOYED: Sat May 30 09:42:26 2026
NAMESPACE: cert-manager
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
cert-manager v1.20.2 has been deployed successfully!

In order to begin issuing certificates, you will need to set up a ClusterIssuer
or Issuer resource (for example, by creating a 'letsencrypt-staging' issuer).

More information on the different types of issuers and how to configure them
can be found in our documentation:

https://cert-manager.io/docs/configuration/

For information on how to configure cert-manager to automatically provision
Certificates for Ingress resources, take a look at the 'ingress-shim'
documentation:

https://cert-manager.io/docs/usage/ingress/

For information on how to configure cert-manager to automatically provision
Certificates for Gateway API resources, take a look at the 'gateway resource'
documentation:

https://cert-manager.io/docs/usage/gateway/
atul@atulrana:~$

```

Command output screenshot

Step 9.5 - Verify cert-manager pods

Command:

```
kubectl get pods -n cert-manager
```

What this does: Checks cert-manager, cainjector and webhook pods.

Result: All cert-manager pods are 1/1 Running.

```

atul@atulrana:~$ kubectl get pods -n cert-manager
NAME                                READY   STATUS    RESTARTS   AGE
cert-manager-5957746d66-zd962       1/1     Running   0           4m19s
cert-manager-cainjector-567c6b47ff-5sczw  1/1     Running   0           4m19s
cert-manager-webhook-7cc5c588cb-mvtm7   1/1     Running   0           4m19s
atul@atulrana:~$

```

Command output screenshot

Step 9.6 - Create Rancher namespace

Command:

```
kubectl create namespace cattle-system
```

What this does: Rancher is installed in the cattle-system namespace.

Result: Namespace cattle-system created.

```

atul@atulrana:~$ kubectl create namespace cattle-system
namespace/cattle-system created
atul@atulrana:~$

```

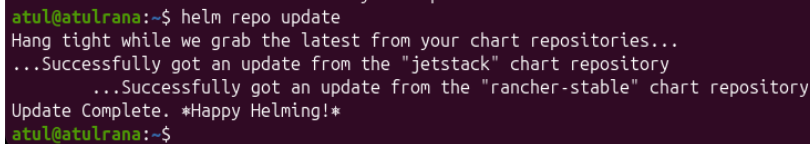
Step 9.7 - Add Rancher Helm repo

Command:

```
helm repo add rancher-stable https://releases.rancher.com/server-charts/stable
helm repo update
```

What this does: Adds Rancher stable Helm chart repository and updates chart index.

Result: Rancher stable repo added and updated.



```
atul@atulrana:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "jetstack" chart repository
...Successfully got an update from the "rancher-stable" chart repository
Update Complete. *Happy Helming!*
atul@atulrana:~$
```

Command output screenshot

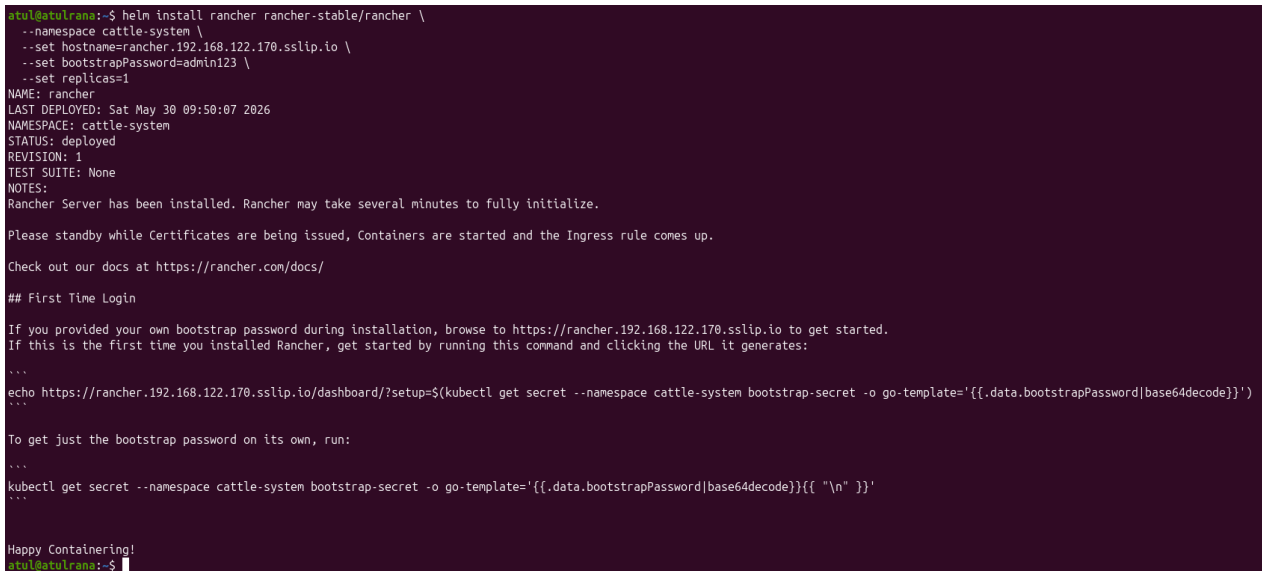
Step 9.8 - Install Rancher

Command:

```
helm install rancher rancher-stable/rancher \
--namespace cattle-system \
--set hostname=rancher.192.168.122.170.sslip.io \
--set bootstrapPassword=admin123 \
--set replicas=1
```

What this does: Installs Rancher server on Kubernetes. The hostname uses sslip.io so DNS resolves to the master IP.

Result: Rancher chart deployed successfully.



```
atul@atulrana:~$ helm install rancher rancher-stable/rancher \
--namespace cattle-system \
--set hostname=rancher.192.168.122.170.sslip.io \
--set bootstrapPassword=admin123 \
--set replicas=1
NAME: rancher
LAST DEPLOYED: Sat May 30 09:50:07 2026
NAMESPACE: cattle-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Rancher Server has been installed. Rancher may take several minutes to fully initialize.

Please standby while Certificates are being issued, Containers are started and the Ingress rule comes up.

Check out our docs at https://rancher.com/docs/

## First Time Login

If you provided your own bootstrap password during installation, browse to https://rancher.192.168.122.170.sslip.io to get started.
If this is the first time you installed Rancher, get started by running this command and clicking the URL it generates:

...
echo https://rancher.192.168.122.170.sslip.io/dashboard/?setup=$(kubectl get secret --namespace cattle-system bootstrap-secret -o go-template='[.data.bootstrapPassword|base64decode]')
...

To get just the bootstrap password on its own, run:

...
kubectl get secret --namespace cattle-system bootstrap-secret -o go-template='[.data.bootstrapPassword|base64decode]{{ "\n" }}'
...

Happy Containering!
atul@atulrana:~$
```

Command output screenshot

Step 9.9 - Verify Rancher pod

Command:

```
kubectl get pods -n cattle-system
```

What this does: Checks Rancher pod status in cattle-system namespace.

Result: Rancher pod reached 1/1 Running.

```
atul@atulrana:~$ kubectl get pods -n cattle-system
NAME                                READY   STATUS    RESTARTS   AGE
helm-operation-zcd4b                0/2     Init:0/1   0           9s
rancher-67c8bfd5b-579dp             1/1     Running    0           3m23s
atul@atulrana:~$
```

Command output screenshot

Step 9.10 - Verify Rancher ingress

Command:

```
kubectl get ingress -n cattle-system
```

What this does: Shows Rancher hostname and ingress addresses.

Result: Host: rancher.192.168.122.170.sslip.io. Ports: 80,443.

```
atul@atulrana:~$ kubectl get ingress -n cattle-system
NAME      CLASS      HOSTS                                ADDRESS                                PORTS   AGE
rancher   <none>     rancher.192.168.122.170.sslip.io    192.168.122.170,192.168.122.193      80, 443 3m55s
atul@atulrana:~$
```

Command output screenshot

Step 9.11 - Ingress controller recovery check

Command:

```
kubectl get pods -n kube-system -o wide | grep ingress
```

What this does: During verification, ingress pods were recreated. This command confirmed the new ingress controller pod became Ready.

Result: rke2-ingress-nginx-controller is 1/1 Running.

```
atul@atulrana:~$ kubectl get pods -n kube-system -o wide | grep ingress
helm-install-rke2-ingress-nginx-s6s9l          0/1     Completed    0           77m    10.42.0.2    atulrana    <none>    <none>
rke2-ingress-nginx-controller-4mtdq            0/1     Evicted      0           15m    <none>      postgres-node3    <none>    <none>
rke2-ingress-nginx-controller-w4fkr            1/1     Running      0           112s   10.42.0.26   atulrana    <none>    <none>
atul@atulrana:~$
```

Command output screenshot

Step 9.12 - Rancher login page

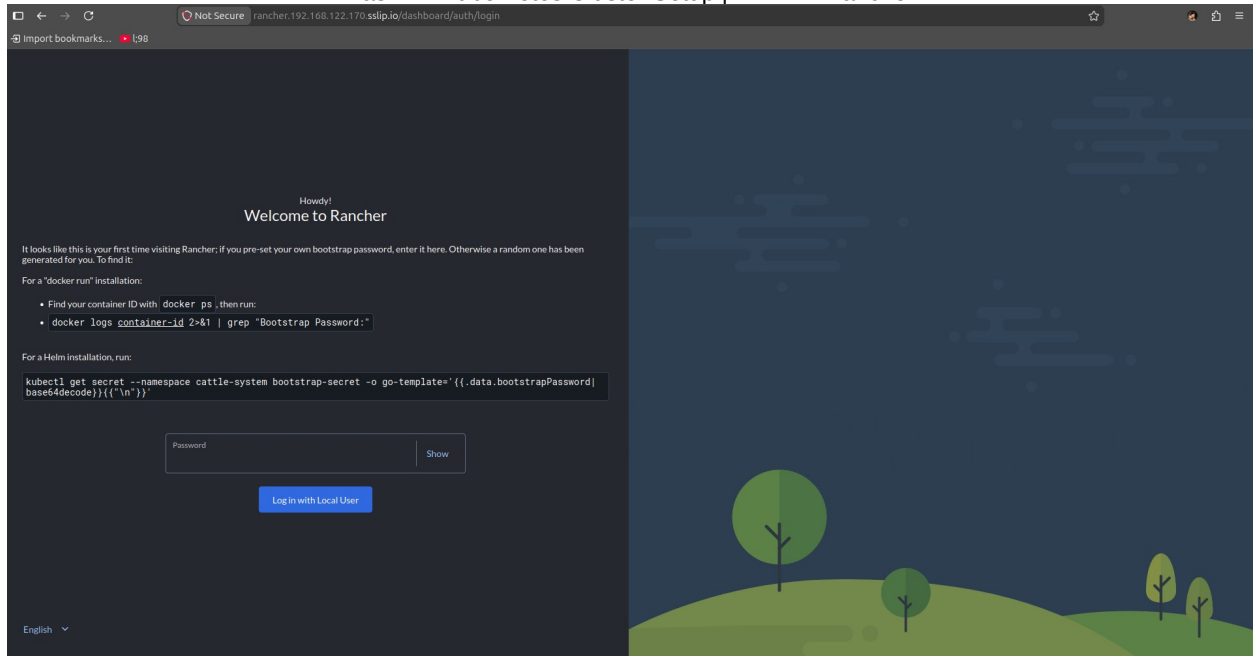
Command:

```
https://rancher.192.168.122.170.sslip.io
```

What this does: Browser access to Rancher UI using configured ingress hostname.

Result: Rancher login page opened successfully.

Task 2 - Kubernetes Cluster Setup | RKE2 + Rancher



Command output screenshot

Step 9.13 - Rancher dashboard

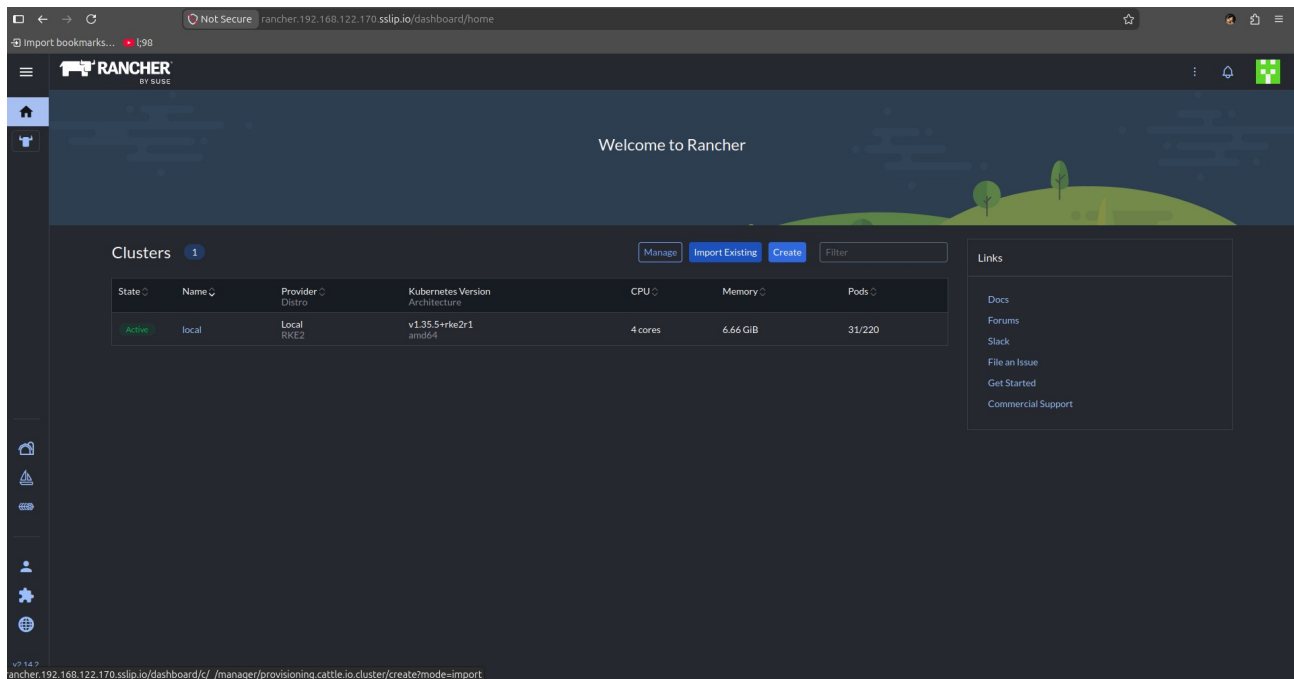
Command:

Login user: admin

Bootstrap password: admin123

What this does: Logged in to Rancher and verified the local RKE2 cluster is visible and active.

Result: Rancher dashboard showed local cluster Active.



Command output screenshot

10. Rancher UI Verification

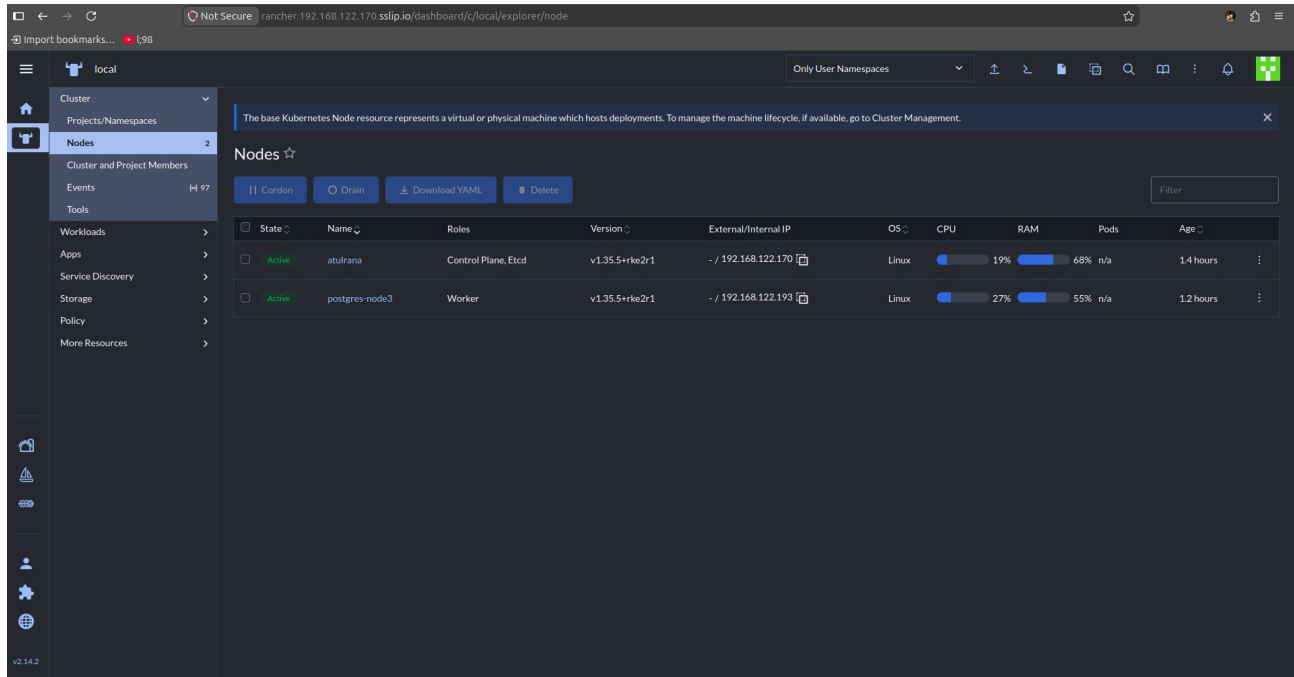
Step 10.1 - Verify nodes in Rancher

Command:

Rancher UI -> local cluster -> Cluster -> Nodes

What this does: Confirms master and worker nodes are visible in Rancher UI.

Result: atulrana Active as Control Plane/Etcd and postgres-node3 Active as Worker.



Command output screenshot

Step 10.2 - Verify nginx pods in Rancher

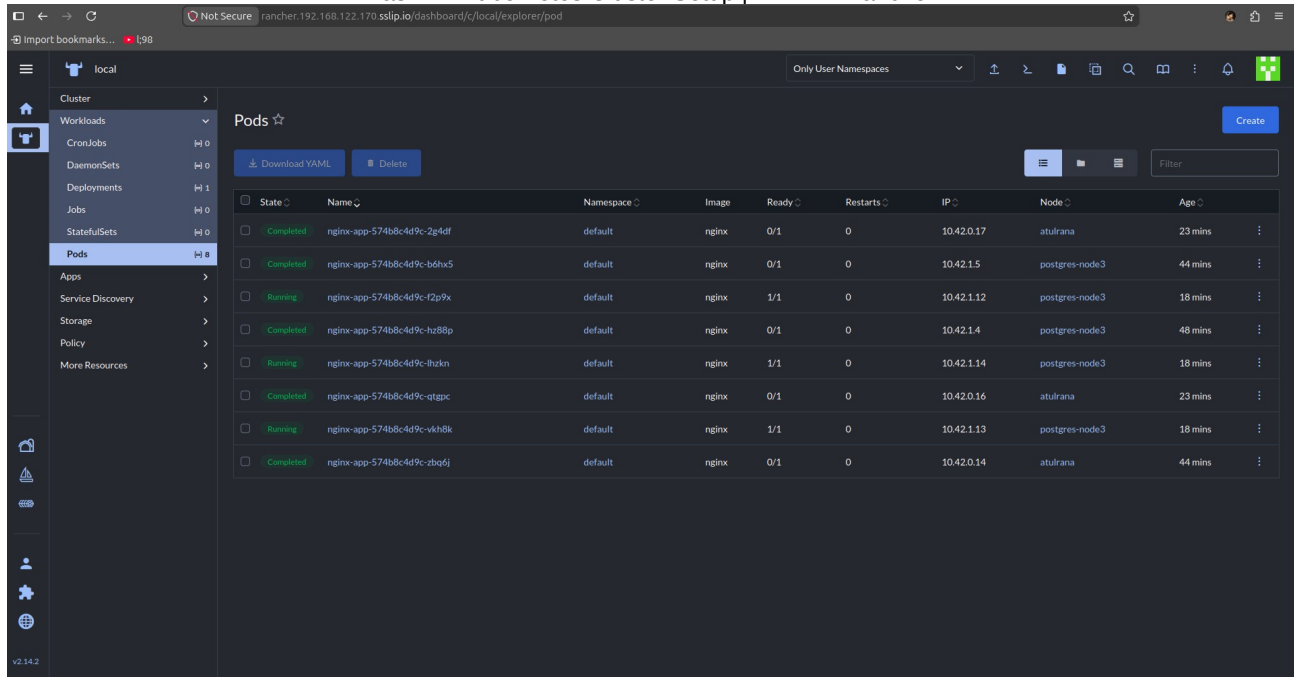
Command:

Rancher UI -> local cluster -> Workloads -> Pods

What this does: Shows nginx pods from Rancher UI. Completed pods are old terminated pods from earlier restarts; active pods are Running.

Result: Three active nginx pods are Running.

Task 2 - Kubernetes Cluster Setup | RKE2 + Rancher



The screenshot shows the Rancher UI interface with the 'Pods' tab selected in the 'Workloads' section. The table displays 10 pods, all in a 'Completed' state, belonging to the 'default' namespace. The pods are named 'nginx-app-574b8c4d9c-2g4df' through 'nginx-app-574b8c4d9c-zbq6j'. The 'Ready' column shows '0/1' for all pods, and the 'Age' column shows various times from 18 to 44 minutes.

State	Name	Namespace	Image	Ready	Restarts	IP	Node	Age
Completed	nginx-app-574b8c4d9c-2g4df	default	nginx	0/1	0	10.42.0.17	atulrana	23 mins
Completed	nginx-app-574b8c4d9c-bdhw5	default	nginx	0/1	0	10.42.1.5	postgres-node3	44 mins
Running	nginx-app-574b8c4d9c-f2p9x	default	nginx	1/1	0	10.42.1.12	postgres-node3	18 mins
Completed	nginx-app-574b8c4d9c-hz88p	default	nginx	0/1	0	10.42.1.4	postgres-node3	48 mins
Running	nginx-app-574b8c4d9c-llzkn	default	nginx	1/1	0	10.42.1.14	postgres-node3	18 mins
Completed	nginx-app-574b8c4d9c-qtspc	default	nginx	0/1	0	10.42.0.16	atulrana	23 mins
Running	nginx-app-574b8c4d9c-vkh8k	default	nginx	1/1	0	10.42.1.13	postgres-node3	18 mins
Completed	nginx-app-574b8c4d9c-zbq6j	default	nginx	0/1	0	10.42.0.14	atulrana	44 mins

Command output screenshot

Step 10.3 - Verify nginx deployment in Rancher and terminal

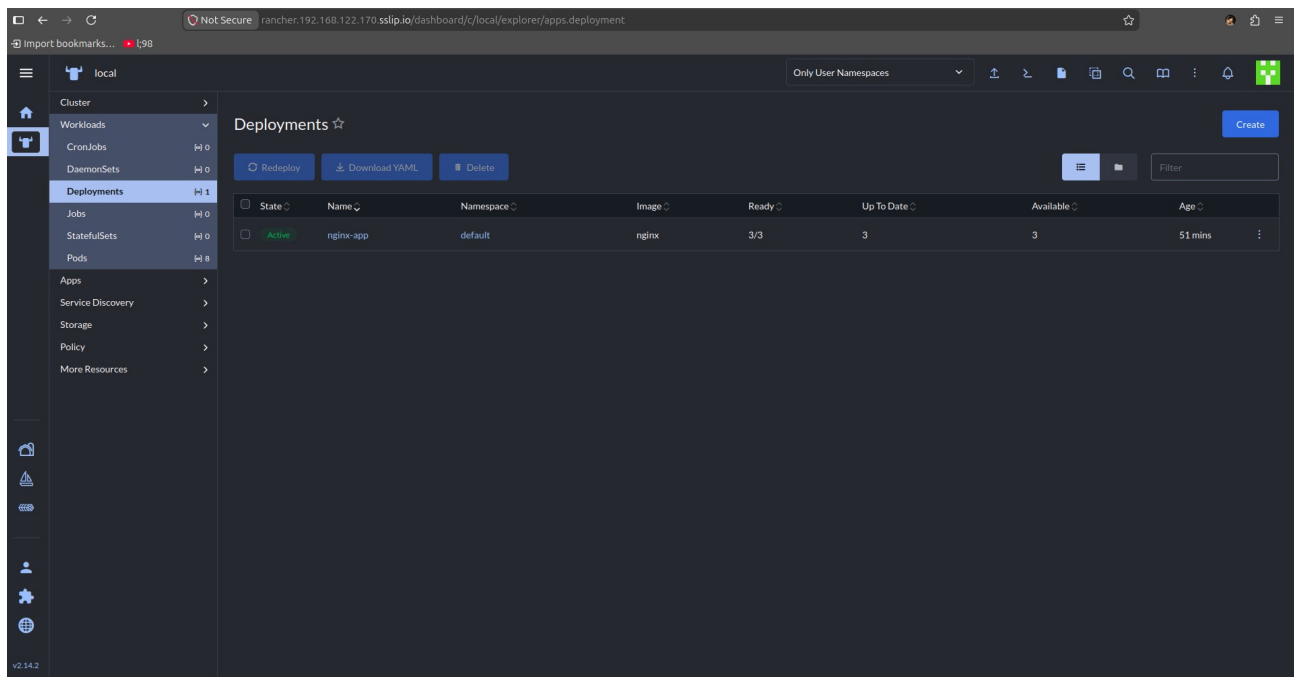
Command:

Rancher UI -> Workloads -> Deployments

kubectl get deployment nginx-app

What this does: Final proof that nginx-app is scaled correctly.

Result: Ready: 3/3, Up To Date: 3, Available: 3.



The screenshot shows the Rancher UI interface with the 'Deployments' tab selected in the 'Workloads' section. The table displays one deployment named 'nginx-app' in the 'default' namespace, using the 'nginx' image. The 'Ready' column shows '3/3', 'Up To Date' shows '3', and 'Available' shows '3'. The 'Age' column shows '51 mins'.

State	Name	Namespace	Image	Ready	Up To Date	Available	Age
Active	nginx-app	default	nginx	3/3	3	3	51 mins

Command output screenshot

11. Important Concepts Explained

Topic	Definition and Importance
RKE2 Server	The master/control-plane component. It runs Kubernetes API server, scheduler, controller-manager, etcd and manages the cluster.
RKE2 Agent	The worker-side service. It connects to the server using server URL and token and runs workloads through kubelet/containerd.
kubectl	Command-line tool used to talk to Kubernetes API server. In this task it was configured on the master node.
Deployment	Kubernetes object that manages pod replicas, self-healing, rolling updates and scaling.
Pod	Smallest deployable unit in Kubernetes. In this task each pod runs an nginx container.
Service	Stable network endpoint for pods. NodePort exposes the service on a port of every cluster node.
Rancher	Web-based Kubernetes management platform for clusters, nodes, workloads, apps, users and monitoring.
cert-manager	Manages certificates inside Kubernetes. Rancher uses it for HTTPS certificates.
Ingress Controller	Reverse proxy that routes external HTTP/HTTPS requests to Kubernetes services.
Scaling	Increasing or decreasing number of pod replicas. Here nginx-app was scaled from 1 to 3 replicas.

12. Final Command Summary

```
# Master: Install RKE2 server
curl -sL https://get.rke2.io | sudo sh -
sudo systemctl enable rke2-server.service
sudo systemctl start rke2-server.service
sudo systemctl status rke2-server --no-pager

# Master: Configure kubectl
mkdir -p ~/.kube
sudo cp /etc/rancher/rke2/rke2.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
export PATH=$PATH:/var/lib/rancher/rke2/bin
kubectl get nodes

# Worker: Install RKE2 agent
curl -sL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sudo sh -
sudo mkdir -p /etc/rancher/rke2
sudo nano /etc/rancher/rke2/config.yaml
sudo systemctl enable rke2-agent.service
sudo systemctl start rke2-agent.service
sudo systemctl status rke2-agent --no-pager

# Nginx deployment and service
kubectl create deployment nginx-app --image=nginx
kubectl expose deployment nginx-app --port=80 --target-port=80 --type=NodePort
kubectl get svc
kubectl scale deployment nginx-app --replicas=3
kubectl get deployment nginx-app
kubectl get pods -o wide
```


Rancher installation

```

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
kubectl create namespace cert-manager
helm repo add jetstack https://charts.jetstack.io
helm repo update
helm install cert-manager jetstack/cert-manager --namespace cert-manager --set crds.enabled=true
kubectl create namespace cattle-system
helm repo add rancher-stable https://releases.rancher.com/server-charts/stable
helm repo update
helm install rancher rancher-stable/rancher --namespace cattle-system --set hostname=rancher.192.168.122.170.sslip.io --set bootstrapPassword=admin123 --set replicas=1
kubectl get pods -n cattle-system
kubectl get ingress -n cattle-system

```

13. Final Verification Summary

Check	Expected Result	Actual Result	Status
RKE2 server	active running	active running	Pass
RKE2 worker	active running	active running	Pass
Nodes	2 nodes Ready	atulrana + postgres-node3 Ready	Pass
Nginx deployment	3/3 replicas	3/3 replicas	Pass
Nginx service	NodePort accessible	Welcome to nginx page opened	Pass
cert-manager	3 pods Running	3 pods Running	Pass
Rancher	UI accessible	Dashboard opened	Pass
Rancher nodes	Master + worker visible	Both Active	Pass
Rancher workloads	nginx visible	nginx-app 3/3 visible	Pass

Final conclusion: The RKE2 Kubernetes cluster was installed successfully with one master and one worker node. Rancher was installed and used to manage/verify the cluster. Nginx was deployed, exposed using NodePort, accessed from browser, and scaled to 3 pods successfully.

Submitted by: Atul Kumar | Task 2 - Kubernetes Cluster Setup | May 2026